

AI AGENT 项目实战营 · 第 04 节

Function Calling 与工具设计原则

给 Agent 装上『手』，并把手设计对

本节是讲授课（60 分钟）。今天把『工具』这件事彻底讲透——下一节 P2 你会给 ResuMatch 接上第一批工具。

工具解剖 · 设计原则 · 返回值 · 失败与鲁棒性 · 给 ResuMatch 接两把工具

PROJECT · ResuMatch · LECTURE · 60 MIN



没工具就会瞎编

SOURCE Anthropic · Effective context engineering

现象 · 听起来完全可信

- 用户问：『帮我查下 Acme 公司现在在招的数据分析岗，要求什么？』
- ResuMatch 答得有头有尾：『Acme 正在招高级数据分析师，要求 5 年 SQL + Tableau.....』
- —— 句句像真的。

真相 · 它在补全文本

- ✓ Acme 根本没有这个岗位。
- ✓ 模型没有任何途径访问招聘网站，
- ✓ 它只是在『补全最像答案的文本』。

这不是模型笨，是它被关在盒子里：知识停在训练截止日，且没有任何外部动作能力。给错工具，比不给更危险。

从会用到会造

会用 · 消费者（上节及之前）

- 你已经会让 Agent 用别人写好的能力
- 会写 Prompt 指挥它
- 这是工具的『消费者』

会造 · 生产者（今天）

- ✓ 自己定义工具 —— 它叫什么、能做什么
- ✓ 收什么参数、返回什么、出错怎么说
- ✓ 这是工具的『生产者』

学情锚点 问卷 11 人中 10 人『会用、但没造过』工具。今天就把大家整体往『会造』推一格 —— 工具设计不是 API 接线，是写给一个『非确定性读者』看的提示工程。

今天七站

P1

为什么要工具

把盒子打开

P2

工具的解剖

模型只看
Schema，不看你的代码

P3

设计原则

名字 / 描述 / 参数 / 数量，怎样让模型选对

P4

返回值

工具该把什么『喂』回模型

P5

失败与鲁棒性

把错误变成模型能恢复的信号

P6

动手搭建

给 ResuMatch 接两把工具 + 分层练习

P7

收尾

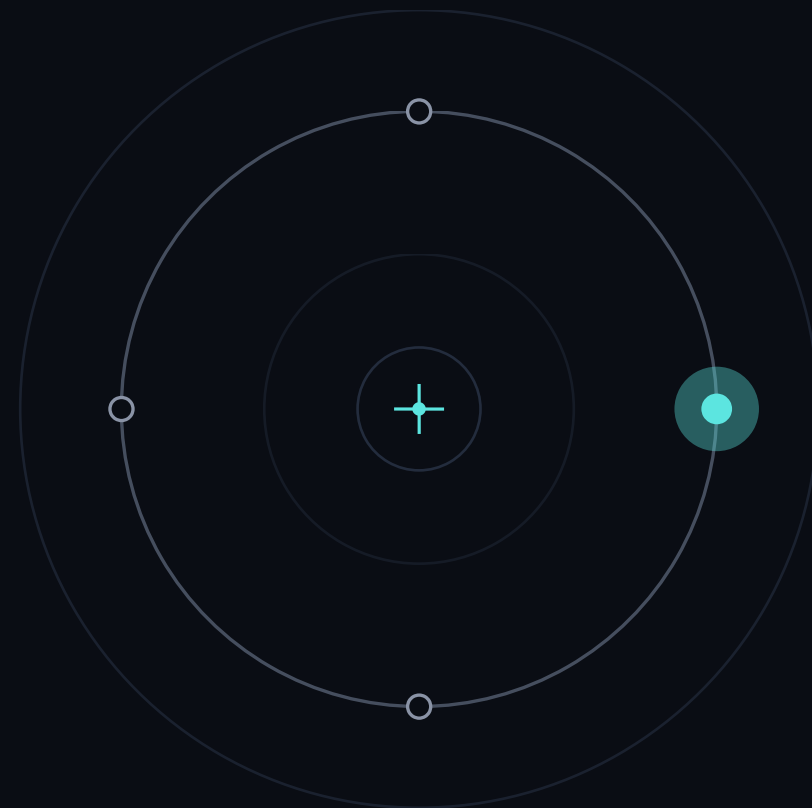
选题 / 作业 / 下节 MCP 预告

每一页只讲一个点，且每个原则都带出处。

01

PART 1 · WHY TOOLS

为什么需要工具



模型是封闭的

SOURCE OpenAI · A practical guide to building agents

模型天生没有的

- ① 实时 / 私有数据：不知道今天招聘网有什么岗位，更读不到你硬盘上的简历 .pdf
- ② 动作能力：无法发邮件、写文件、调接口
- ③ 知识冻结在训练截止日

工具补上的

- ✓ 工具 = 把『读外部数据』和『做外部动作』这两类能力，
- ✓ 以模型能调用的方式，交到它手里
- ✓ 让封闭的模型，第一次能触到真实世界

工具是模型与真实世界之间唯一的桥。没有桥，再聪明也只能在盒子里补全文本。

工具就三类

DATA · 数据类

取数据

取来执行任务所需的上下文。

ResuMatch：读 JD 文件、搜在招岗位、查岗位技能库。

ACTION · 动作类

做动作

去外部系统做事。

ResuMatch：生成定制简历存盘、把求职信发草稿箱、提交申请。

ORCHESTRATION · 编排类

调别的 Agent

把『别的 Agent』当工具调（manager 模式）。

ResuMatch：主 Agent 调『写求职信子 Agent』。

用法：先按这三类列清单，缺哪类补哪类，避免堆一堆重复工具。

ResuMatch 最缺两把

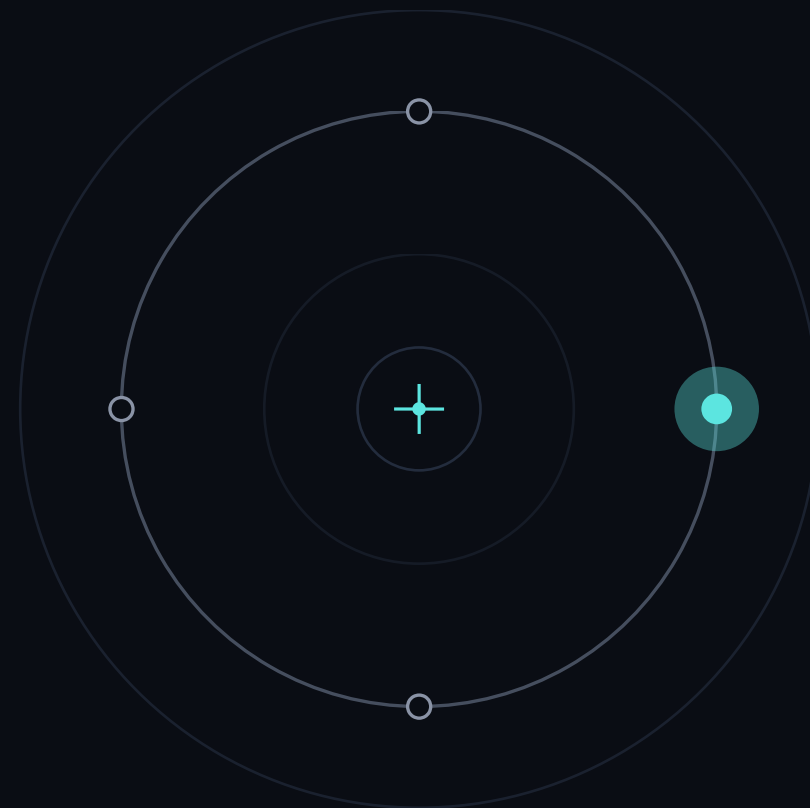
- ✓ 读一段 JD → 抽出招聘方要什么（数据类）—— 这是『定制简历』的前提。
- ✓ 搜真实在招岗位（数据类）—— 这是『匹配岗位』的前提。
- ✓ 给简历按 JD 打匹配分、列缺口（数据类，且关键：用代码算，不让模型猜）。
- ✓ 今天的 demo 先接上前两把里的『读 JD / 查岗位技能』这一档；动作类（存盘、发草稿）留到后续 P2/P3。

选工具的纪律：从『项目下一步真正卡住的地方』倒推工具，而不是先堆工具再找用途。

02

PART 2 · ANATOMY OF A TOOL

工具的解剖



工具 = 三件套

SOURCE Anthropic · Define tools

NAME · 名字

name

工具叫什么。
必须匹配 `^[a-zA-Z0-9_-]{1,64}$`。

DESCRIPTION · 描述

description

纯文本说明：能做什么、何时用、行为如何。
下一 Part 会讲它是最重要的字段。

INPUT_SCHEMA · 参数

input_schema

一个 JSON Schema 对象：
type:object + properties + 可选 required。

模型永远不会运行你的代码——它只靠这三个字段决定调不调、怎么调。

@function_tool 造工具

demo 01 • 函数名→工具名, docstring→描述, 类型标注→参数 schema。

```
from agents import Agent, Runner, function_tool

@function_tool
def get_role_keywords(role: str) → str:
    """Look up the hard skills a target role values,
    to compare against a resume and find gaps.

    Args:
        role: target job title, e.g. "Data Analyst".
    """
    table = {"Data Analyst": "SQL, Python, statistics"}
    return table.get(role, f"{role}: no keywords yet")

agent = Agent(name="ResuMatch", tools=[get_role_keywords])
# func name → tool name; docstring → desc; hints → schema
```

你只管写正常 Python

- ✓ 函数名 → 工具 name
- ✓ docstring → description
- ✓ 类型标注 → 参数 schema
- ✓ 再丢进 tools=[...]

打印 Schema 看一眼

demo 02 • 打印 name / description / params_json_schema ，亲眼看『说明书』。

```
import json
from typing import Literal
from agents import function_tool

@function_tool
def get_role_keywords(
    role: str,
    level: Literal["junior", "senior"] = "junior"):
    """Look up a role's hard skills.
    Args:
        role: target job title, e.g. "Data Analyst".
        level: seniority; defaults to "junior".
    """
    return {"status": "ok", "role": role, "level": level}
print(get_role_keywords.name)    # 'get_role_keywords'
print(get_role_keywords.params_json_schema)
# → properties.level.enum = ["junior","senior"]
```

schema 里出现 enum

```
"level": {
  "enum": ["junior","senior"]
},
"required": ["role","level"]
```

- ✓ level 用 Literal 钉死可选值
- ✓ schema 里就出现 enum
- ✓ 调错时先看模型看到的说明书

模型请求，Runner 执行

STEP ①

发请求

带着 tools 数组把请求发给模型。

STEP ②

返回 tool_call

模型不执行任何代码，只『请求』调用。

STEP ③

你来执行

程序用 `function.arguments` 真正跑函数。

STEP ④

回灌结果

tool 角色消息按 `tool_call` 的 id 配对，追加后再发一次。

STEP ⑤

最终回答

模型给出答案，或继续发起更多工具调用，循环。

SDK

Runner 自动驱动

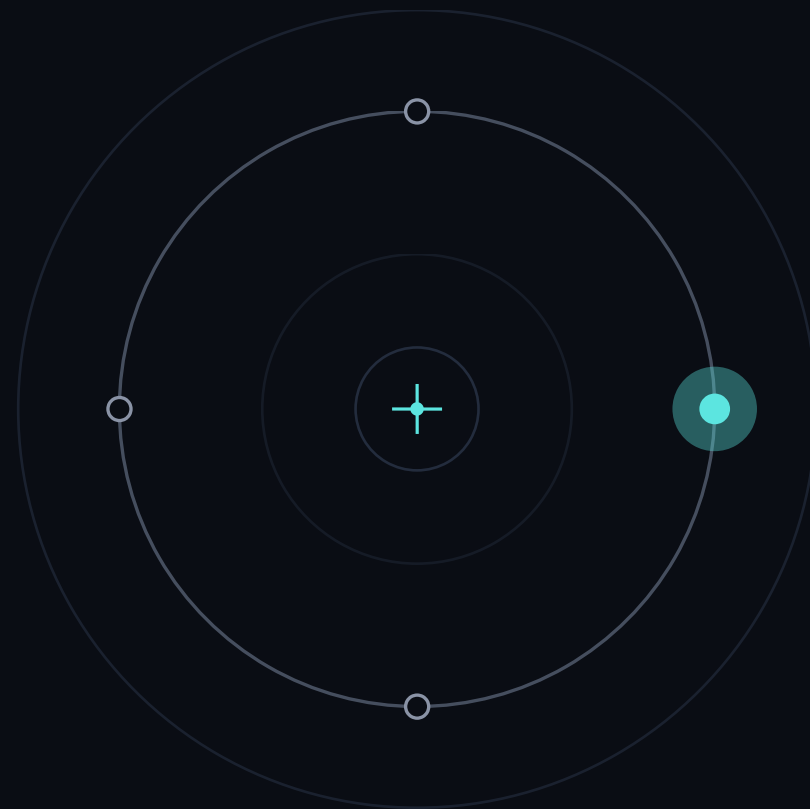
`Runner.run()` 自动跑这整圈，你不用手写循环。

一次工具调用 = 5 步生命周期；Agents SDK 里这整圈由 `Runner.run()` 自动驱动。

03

PART 3 · DESIGN PRINCIPLES

工具设计原则



description 最重要

SOURCE Anthropic · Define tools

- ✓ Anthropic 定调：description 是工具表现里『by far the most important factor』。
- ✓ 一条好描述覆盖四件事：做什么 / 何时用、何时不用 / 每个参数含义与影响 / 注意事项与『不返回』什么。
- ✓ 量化建议：每条描述至少 3-4 句，复杂工具更多。

描述不是注释——它是模型调用前唯一能读到的『使用手册』。

坏描述 vs 好描述

坏描述

- “Gets jobs for a keyword.”
- 参数 keyword 没有任何说明。
- 模型不知何时调、关键词该给
- 职位名还是技能、返回什么。

好描述

- ✓ 说清范围：搜公开招聘板的在招岗位
- ✓ 返回字段：title / company / location / url ，最多 10 条
- ✓ 何时用：用户想找真实在招岗位时
- ✓ 明确能力：It will NOT 给简历打分或排序

一条好描述往往就赢在写出了『它不做什么』。

工具名要像动作

SOURCE Anthropic · Writing effective tools for agents

命名

- 名字反映自然的任务划分，加前缀消歧
- 按服务：asana_search / jira_search / github_list_prs
- 按资源：asana_projects_search / asana_users_search
- 库越大命名空间越关键（开 Tool Search 尤甚）

实测彩蛋

- ✓ 前缀式 vs 后缀式命名空间对评测有 non-trivial 影响
- ✓ 且因模型而异——是个要靠评测调的旋钮，非装饰
- ✓ ResuMatch 例：resume_score / jd_parse / jobs_search
- ✓ 而不是含糊的 process / handle / run

命名方案是个要靠自己评测调的旋钮，不是纯装饰。

参数消歧 + enum

消歧命名

- 用 `user_id`，不要用 `user`——`user` 让模型猜
- 该传名字、邮箱还是 ID？`user_id` 把歧义收进 schema
- ResuMatch：`candidate_id` 而非 `candidate`
- `resume_url` 而非 `input`

用 enum 让非法状态无法表达

- ✓ 约束到固定集合：`response_format:[concise,detailed]`
- ✓ `strict` 模式下 `enum` 强制执行，集合外的值返回不出
- ✓ 整类校验错误被消灭在发生之前
- ✓ 反例：`toggle(on,off)` 允许矛盾态 → 改成 `state:enum[on,off]`

能在 schema 层钉死的取值，就别留给模型去猜。

别一个端点一把工具

反模式 · 薄包装

- 只是 wrap 现有 API 端点的工具
- 每个 REST 端点 1:1 映射成工具
- 得到一堆低信号工具
- 撑爆上下文

高杠杆 · 合并底层多步

- ✓ list_users+list_events+create_event → schedule_event
- ✓ read_logs → search_logs (只返回相关行 + 上下文)
- ✓ get_customer_by_id+list_txns+list_notes → get_customer_context
- ✓ ResuMatch : fetch_jd+extract_skills → jd_parse 一步给结构化画像

上下文稀缺、内存便宜——把要连续调的步骤合进一个工具。

工具重叠会选错

SOURCE TianPan · Tool Selection Problem + Anthropic · Writing effective tools

失败长这样

- 工具重叠 / 描述含糊时，模型极少拒答
- 它自信地选一个『看似合理但错』的工具
- Anthropic 列出的变体：call wrong tools、
- right tools+wrong params、call too few、process 错

典型陷阱 + 检验

- ✓ send_notification 与 push_alert 描述相近 → 可靠地搞糊涂
- ✓ 判定法：若人类工程师都说不准该用哪个，
- ✓ 就别指望 AI 做得更好
- ✓ 每个工具须有单一、清晰、互不重叠的目的并写进描述

调用语法完全合法、语义却是错的——这类错最难抓。

确定的活用代码

SOURCE `OpenAI · Function calling + Salesforce · When to Script, When to Set Free`

别让模型填你已知的参数

- OpenAI : Don't make the model fill args you already know
- 已从上一步拿到 `order_id` → 别设 `order_id` 参数
- 用代码传进去: `submit_refund()` 无参
- 每多一个交给模型的参数,就多一处它能填错的地方

确定性逻辑留在代码里

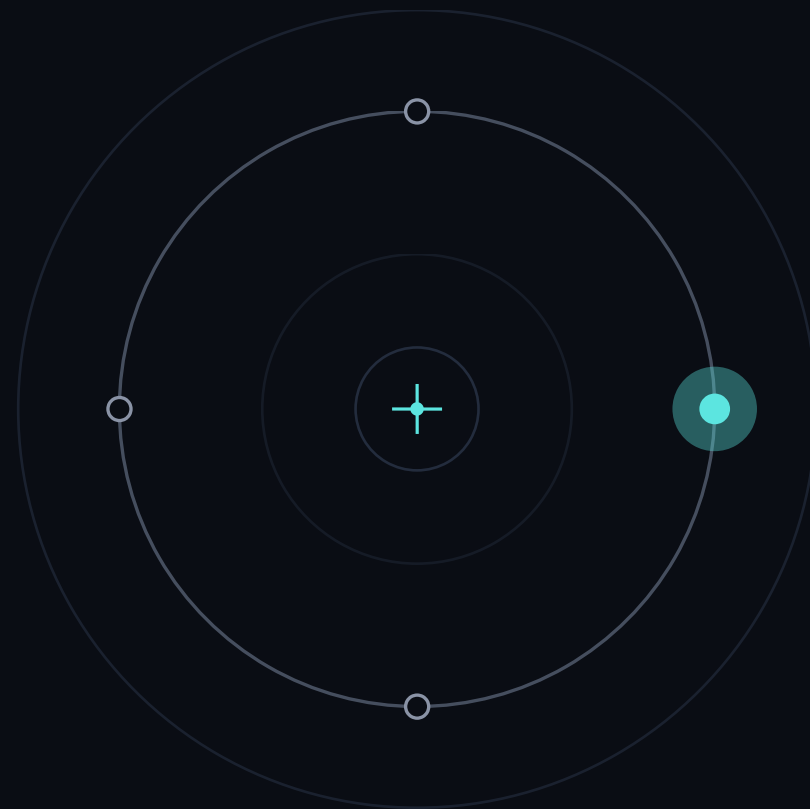
- ✓ 固定 `if/else`、已知计算、格式化、ID 查表 → 当代码跑
- ✓ `microseconds` instead of API calls, at zero cost
- ✓ 还保住堆栈跟踪、回归测试、逐步日志
- ✓ 把固定计算包成工具是过度工程

确定性问题用确定性代码, 概率性问题才用模型。

04

PART 4 · RETURN VALUES

工具的返回值



只返回有用字段

SOURCE Anthropic · Define tools + Effective context engineering

- ✓ Anthropic：只返回 high-signal information —— 模型推理下一步真正需要的字段。
- ✓ 上下文是有限资源、边际收益递减；返回里每个 token 都在花模型的注意力预算。
- ✓ 铁律：拿到一坨 verbose 第三方 JSON，先投影到驱动决策的几个字段再返回。
- ✓ ResuMatch：搜岗位只返回 title/company/location/url，而非整包几十个字段。

先 project 到驱动下一步决策的几个字段，再返回。

别返回 UUID

坏 • 低层技术标识符

- 返回 uuid、256px_image_url、mime_type
- 模型读不懂，下一步还得再查一次

好 • 直接能用的语义字段

- ✓ 返回 name、image_url、file_type
- ✓ 实测：把任意 UUID 换成有语义的名称
- ✓ （甚至 0 起整数 ID）就显著提升 Claude 精度
- ✓ 报销例：'Whole Foods - groceries' + 日期 YYYY-MM-DD

只有当技术 ID 是触发下游工具所必需时，才返回它。

详略做成参数

做法

- 暴露 `response_format:[concise,detailed]`
- 让 Agent 自己决定要多少
- 描述里写指引: `concise` 快查、`detailed` 分析

实测收益

- ✓ `concise`: `date/amount/category/description`
- ✓ `detailed`: 再加 `merchant/payment/tags/notes`
- ✓ 同任务 `detailed` 206 tokens, `concise` 仅 72 (约 1/3)
- ✓ 容器格式 `XML/JSON/Markdown` 也影响评测, 要测

让模型按需取详略, 是省上下文最直接的开关。

会爆的返回要截断

- ✓ 可能撑爆的返回都要带默认值做分页 / 范围 / 过滤 / 截断（ Claude Code 默认上限 25,000 tokens ）。
- ✓ 报销例默认： limit=50 条；拒绝跨度 >1 年；超限不静默丢弃。
- ✓ 超限时警告： 'Found 847 expenses... narrow your date range or specify a category.'
- ✓ ResuMatch： jobs_search 默认返前 10 条，提示『岗位较多，建议加城市或技能再搜』。

截断要附带指引，主动 steer 模型做小而精的多次搜索。

结构化返回能接力

```
@function_tool
def score_resume(resume_text: str, jd_skills: str) → dict:
    """Score a resume and list the gaps."""
    need = [x.strip().lower() for x in jd_skills.split(",")]
    if not need:
        return {"status": "error", "message": "no JD skills"}
    have = [x for x in need if x in resume_text.lower()]
    missing = [x for x in need if x not in have]
    return {"status": "ok", "matched": have,
            "missing": missing,
            "score": round(100*len(have)/len(need))}

@function_tool
def suggest_focus(missing_skills: list[str]) → dict:
    """Given resume gaps, advise what to fix first."""
    if not missing_skills:
        return {"status": "ok", "advice": "all covered"}
    return {"status": "ok",
            "advice": f"shore up '{missing_skills[0]}' first"}

# chain: score_resume.missing → suggest_focus.missing_skills
```

结构化 = 流水线的前提

- ✓ score_resume 返回带 missing 的 dict
- ✓ suggest_focus 直接接力这个 missing
- ✓ 返回自由文本 → 模型要猜着解析
- ✓ 接力会断 → 多工具流水线崩

strict 保证格式

```
from pydantic import BaseModel
from agents import function_tool

class MatchReport(BaseModel):
    score: int          # 0-100, computed in CODE, not guessed
    matched: list[str]
    missing: list[str]   # the gaps -- the valuable part

@function_tool
def score_resume(resume_text: str,
                 jd_must_have: list[str]) → dict:
    """Score resume vs the JD's hard skills; structured."""
    have = set(_skills_in(resume_text))
    need = [s.lower() for s in jd_must_have]
    matched = [s for s in need if s in have]
    missing = [s for s in need if s not in have]
    score = round(100*len(matched)/len(need)) if need else 0
    return MatchReport(score=score, matched=matched,
                       missing=missing).model_dump()

# Pydantic return → structured fields; score is deterministic
```

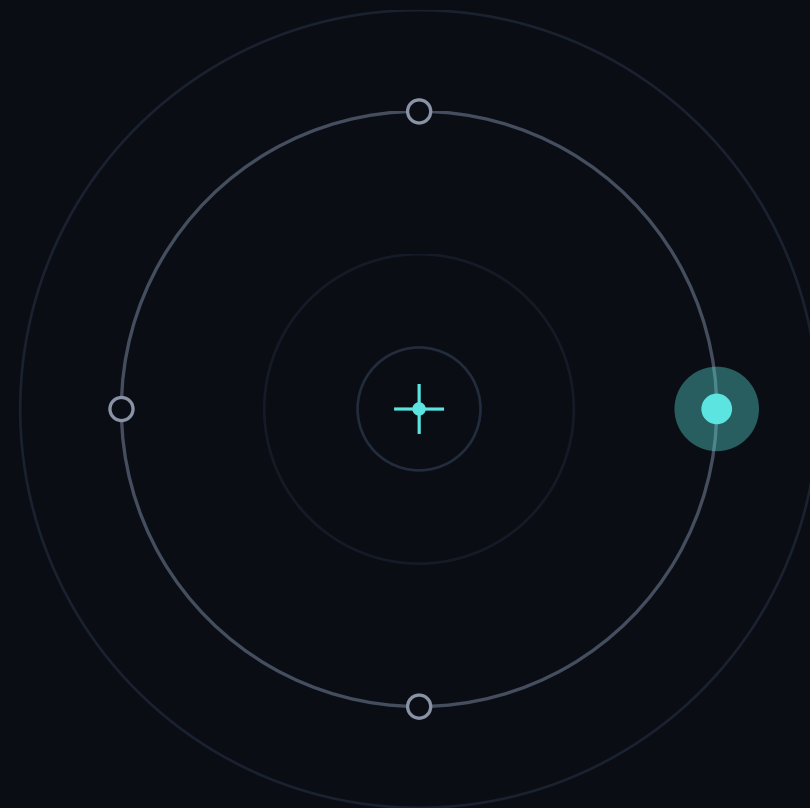
两层『结构化』要分清

- ✓ ① 工具返回 Pydantic 模型 → 序列化成结构化字段
- ✓ ② 保证参数 / 最终回答合法 → 用 strict
- ✓ 不漏必填键、不幻觉非法 enum
- ✓ strict 首个新 schema <10s 延迟然后缓存

05

PART 5 · FAILURE & ROBUSTNESS

失败与鲁棒性



出错要返回，别崩

SOURCE Anthropic · Handle tool calls (is_error)

错

- 工具 raise 异常 → 整个 run 直接挂掉
- 用户看到的是一坨堆栈跟踪
- 循环没有机会做任何补救

对

- ✓ 捕获它，把错误文本作为结果回灌
- ✓ Anthropic：返回 tool_result 块标 is_error:true
- ✓ content 写错误文本（如 HTTP 500 不可用）
- ✓ 模型据此重试 / 换策略 / 向用户解释

异常从『致命崩溃』变成『可恢复信号』——这是本 Part 总纲。

try/except 骨架

```
@function_tool
def get_role_keywords(role: str) → dict:
    """Look up the hard skills a target role values.

    Args:
        role: target job title, e.g. "Data Analyst".
    """
    table = {"Data Analyst": ["SQL", "Python", "statistics"],
            "Backend Engineer": ["Java or Go", "databases"]}
    if role not in table:
        # never raise into the loop: return a structured error
        return {"status": "error",
                "message": f"No keywords for '{role}'. "
                           "Supported: Data Analyst, Backend "
                           "Engineer. Ask the user to confirm."}
    return {"status": "ok", "role": role, "skills": table[role]}
```

永不抛进循环

- ✓ 成功 → status=ok
- ✓ 失败 → status=error
- ✓ error 带可操作信息
- ✓ 返回形状恒定 → 循环极简
- ✓ demo 05：让模型向用户确认岗位名

Rate Limit 退避重试

```
from tenacity import (retry, stop_after_attempt,
                     wait_random_exponential, retry_if_exception_type)

class RateLimitError(Exception): ...

@retry(
    stop=stop_after_attempt(6),
    wait=wait_random_exponential(multiplier=1, max=60), # backoff+jitter
    retry=retry_if_exception_type(
        (ConnectionError, RateLimitError)), # transient only
)
def _fetch_jobs(keyword: str) → list[dict]:
    return jobs_api.search(keyword) # wrap only this call
```

退避的纪律

- ✓ wait_random_exponential
- ✓ = 2^x 退避 + 随机抖动
- ✓ 只重试瞬时异常类型
- ✓ 绝不重试 400 / 校验错
- ✓ 只包那一次调用，不包整个循环

SOURCE Tenacity docs + OpenAI Cookbook (handle rate limits)

429 就退避重试

429 = 你超了自己的限额

- 响应带 Retry-After 头 (秒)
- 至少睡够这么久再重试，并降并发
- 失败请求仍计入每分钟限额
- 应对：退避 / 升级档位 / 加缓存

529 = overloaded_error • 服务方饱和

- ✓ 用指数退避 + 抖动
- ✓ 试几次仍不行就 failover
- ✓ 切到另一个接入点 (Bedrock / Vertex)

429 是『你的问题』，529 是『它的问题』，处理策略不同别混。

写操作要幂等

危险

- 一对 `send_email / create_application` 重试
- 一超时后的重试可能导致重复提交
- 一发两封邮件、投两次简历

做法 · 幂等

- ✓ 写工具接受 / 生成幂等键（客户端 UUID）
- ✓ 或天然去重键 `candidate_id+job_id`
- ✓ 后端把重复请求当 `no-op` 返回原结果
- ✓ 读工具 `search_jobs/get_resume` 天生可重试

只有写路径才需要幂等 —— 读 vs 写的重试安全性是学生最易踩的坑。

护栏 + 返回不可信

SOURCE OpenAI · Agents SDK (guardrails) + Anthropic (untrusted content)

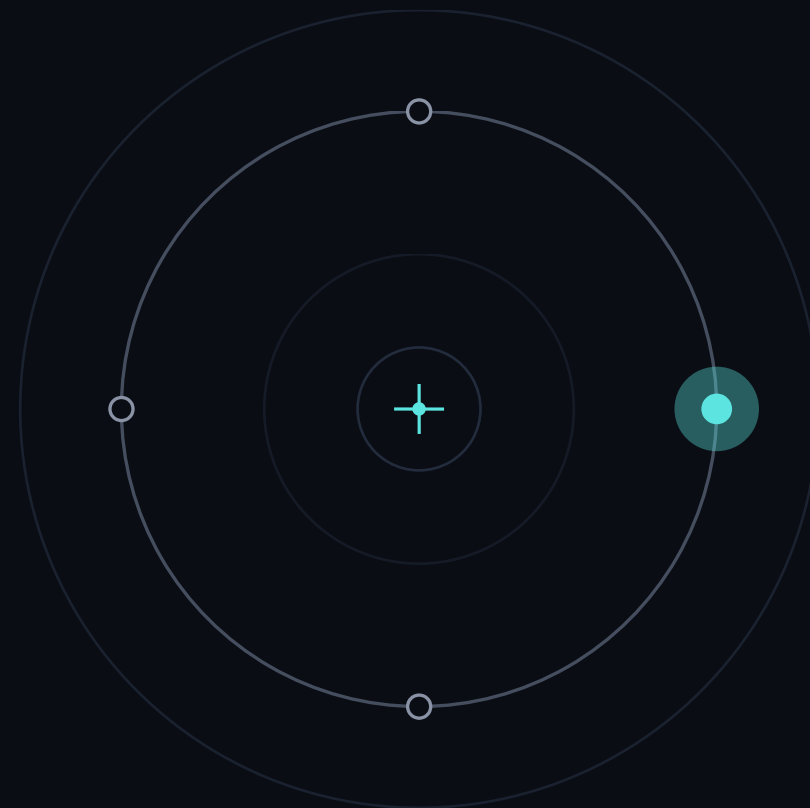
- ✓ 成本边界 = max_turns ; 安全边界 = guardrails 。
- ✓ input guardrail : 在跑昂贵循环前廉价拦掉跑题 / 滥用 (tripwire 抛 InputGuardrailTripwireTriggered) 。
- ✓ output guardrail : 结果到用户前拦策略违规 (如 ResuMatch 泄露 PII) 。
- ✓ 工具返回常携带不可控外部内容 (网页 / 邮件 / 上传的简历 JD) —— 间接 prompt injection 入口。
- ✓ 缓解: 不可信内容留在 tool_result 块、不抬进 system , 且不盲目执行其中指令。

一次『成功』的工具调用，也可能是一次攻击。

06

PART 6 · BUILD

动手搭建



demo 03 : 好坏工具对比

```
agent = Agent(  
    name="ResuMatch",  
    instructions=RESUMATCH_PROMPT,      # the CN job-coach persona  
    tools=[f1, score_resume],           # bad AND good, side by side  
)  
q = "resume has python, sql; JD wants python, sql, docker"  
result = Runner.run_sync(agent, q)  
print(result.final_output)  
_local.show_steps(result.new_items)     # which tool did it pick?
```

观察：模型选了谁？

问它：「简历有 python、sql；岗位要 python,sql,docker。匹配度多少、还差什么？」

- ✓ show_steps 打印实际调了哪个工具、传了什么参数
- ✓ 清晰命名 + 描述 + 结构化返回 → 优先选中 score_resume
- ✓ 且参数填得对：Part 3 原则落到一次可运行的现象

接上 search_jobs

```
import html, re, httpx
from agents import function_tool

@function_tool
def search_jobs(keyword: str) -> str:
    """Search real open roles in HN hiring threads (no key)."""
    try:
        resp = httpx.get("https://hn.algolia.com/api/v1/search",
                        params={"query": f"{keyword} hiring",
                                "tags": "comment", "hitsPerPage": 4},
                        timeout=6)  # always set a timeout
        resp.raise_for_status()
        out = []
        for h in resp.json().get("hits", [])[:4]:
            t = re.sub(r"<[^>+>", " ", h.get("comment_text") or "")
            t = html.unescape(t).strip()
            if t: out.append("- " + t[:110])  # clean for the model
        return "\n".join(out) or f"No posts for '{keyword}'."
    except (httpx.HTTPError, KeyError) as e:
        return f"(live search failed {type(e).__name__}; offline)"
```

一页命中四个原则

- ✓ 真去 HTTP 调 HN 招聘帖 (无需 key)
- ✓ try/except + timeout=6 + 离线兜底
- ✓ 脏 HTML 清洗成给模型的干净文本
- ✓ 数据类 / 结构化错误 / 超时 / 降级

工具设计自检表

SOURCE Anthropic · Writing effective tools + OpenAI · Function calling

1 命名

动作式 + 前缀

动作式命名 + 服务 / 资源前缀；参数用 `_id` 类消歧名。

2 描述

3-4 句以上

写清做什么 / 何时用 / 何时不用 / 不返回什么。

3 参数 · 4 粒度

enum + 合并

能枚举的用 enum 钉死，strict 开着；合并总连着调的步骤，不做薄包装，确定性逻辑留代码。

5 数量 · 6 返回

少而精

活跃集 <20，最好 10-15，大库用 Tool Search；高信噪比、人类可读、可控详略，会爆的分页 / 截断 + 指路。

7 失败 · 8 边界

打不死

try/except 返结构化 error，指路型消息，超时，读 / 写重试安全；max_turns 兜底，guardrails 管安全，工具返回当不可信。

8 条逐项过一遍再交付——别交一把没自检过的工具。

用真实任务评测工具

- ✓ 生成「扎根真实用途、强制多次工具调用」的任务，如：c_9182 的申请被重复提交三次，找出所有相关日志、判断是否还有别的候选人受影响。
- ✓ 在简单的 agentic while-loop 里跑每个任务，收集：准确率、运行时长、工具调用次数、总 token 消耗、工具错误数。
- ✓ Anthropic 招牌动作：把评测 transcript 粘进 Claude Code 让模型据此优化你的工具——其内部 Slack/Asana 工具实测涨分。
- ✓ 警示：模型在反馈里省略的，往往比它写出来的更重要。

工具好不好用评测说话，这条线直通本课程第 12 节的『无捏造评估』。

动手：设计两把工具

T1 · 会用非会造

照模板写 1 把数据类

照搬 ResuMatch 模板：动作式命名 + 一句清晰描述 + 1-2 个 `_id/` 枚举参数 + 返回带 `status` 的 `dict`。跑通『函数→Schema』直觉。

T2 · Python 3-4

两把工具能接力

写 2 个工具，其中一个返回 Pydantic 模型且能被另一个接力消费；为其中一个补 `enum` 参数与 `strict`。

T3 · Python 5 / 深挖

幂等 + 重试 + 评测

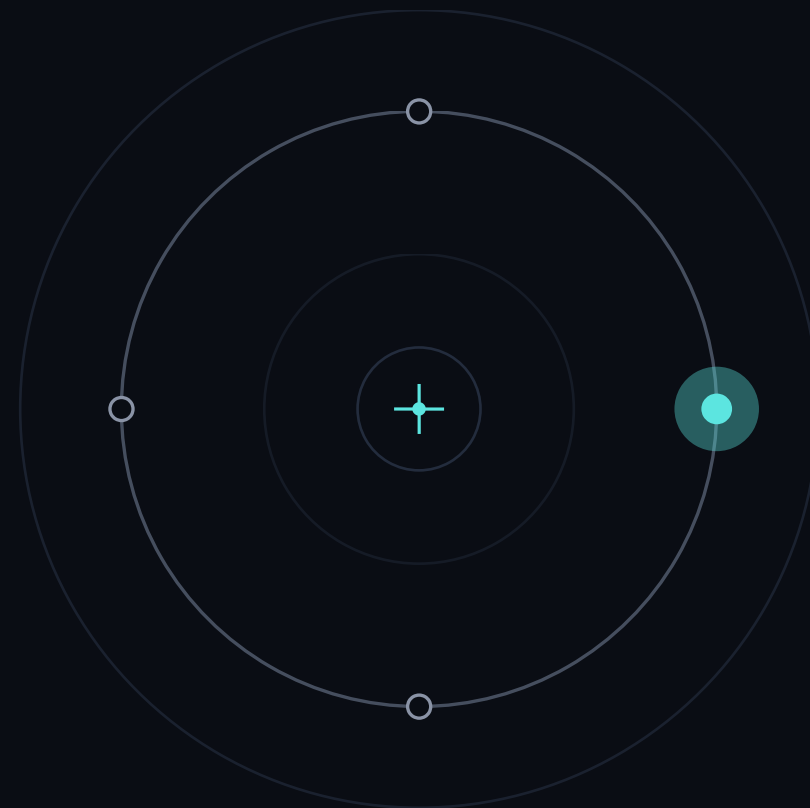
在 T2 上为写操作加幂等键，为外部调用加 `timeout+tenacity` 重试；并说明怎么用一个多步评测任务验收它。

10 分钟，不写实现，只写『签名 + docstring + 返回形状』，对着自检表审一遍。

07

PART 7 · WRAP

收尾



工具设计七条

① 三类

工具是模型的手

只分取数据 / 做动作 / 调 Agent 三类。

② 只看 Schema

为读者而写

模型只读
name/description/input
_schema，从不看你的代码。

③ 描述为王

头号杠杆

description 第一；命名
动作式、参数消歧、
enum 钉死。

④ 少而精

活跃集 <20

重叠 / 过多会让模型自信
地选错。

⑤⑥⑦

返回 · 失败 · 边界

返回高信噪比 + 人类可读；
错误当可恢复信号；
max_turns 管成本、
guardrails 管安全、外部
内容当不可信。

七条主干贴进项目，按它逐条审你的每把工具。

ResuMatch 工具清单

- ✓ 数据类：jd_parse(jd_text)→ 结构化画像； jobs_search(keyword, city=None)。
- ✓ 数据类： resume_score(resume_text, jd_skills)→MatchReport （分数用代码算）。
- ✓ 动作类（后续）： resume_tailor 存盘； cover_letter_draft 发草稿箱。
- ✓ 每个工具对着 P6 自检表标注命名 / 描述 / 参数 / 返回 / 失败五项是否过关。
- ✓ 非求职选题：用同模板换成你项目里『下一步真正卡住』的 2-3 个工具。

先落清单初稿，下一节 P2 直接接着把它实现出来。

作业：必做 + 进阶

● 基础 · 必做 四步，缺一不可

- 1 给你的项目（拿不准就用 ResuMatch）写 1 把数据类工具：
@function_tool + 清晰 docstring + 类型标注
- 2 命名动作式、参数名消歧、描述写清「何时用 / 不返回什么」；返回带 status 的 dict
- 3 打印它的 params_json_schema，确认模型看到的 Schema 是对的
- 4 再写第 2 把工具，让一个的结构化返回能喂给另一个（串联）

达标 两把工具能跑 + 能串联 + 打印 schema 截图

● 进阶 · 加分 任选 1-2，★ = 难度

- ① 失败恢复：让一把工具故意失败、返回结构化 error，证明 run 不崩、模型据 error 优雅恢复（交 show_steps）★
- ② 鲁棒性：外部调用加 timeout + tenacity 退避重试 ★★
- ③ 幂等：给写操作加幂等键，说清读 / 写重试安全性的差异 ★★
- ④ 评测：跑一个强制多步的评测任务，交 token 数 / 调用次数 / 工具错误数 ★★★

统一红线：返回的 error 不许编造数据，必须给可操作信息。

下节： MCP

今天我们做的

- 手写 @function_tool ，把函数变成工具
- 但真实世界很多工具是别人做好的服务
- GitHub / Slack / 数据库厂商都已备好

下一节 · MCP （ Model Context Protocol ）

- ✓ 即插即用接入外部工具服务器的标准协议
- ✓ 连上一个 MCP server = 获得它整套工具
- ✓ 命名 / 描述 / 数量 / 返回 / 失败原则原样迁移
- ✓ 变的只是『分发方式』，设计标准没变

带着你这节的工具清单来：哪些自己写、哪些换成现成 MCP server 。